

School of computer science
UNIVERSITY OF PETROLEUM
AND
ENERGY STUDIES



6th semester
Name : Ananya Tiwari
SAP ID : 500119141
Course : Btech CSE (CCVT)
BATCH : 1
SUBJECT : Containerization and DevOps
ASSIGNMENT 1

PROJECT REPORT

1. Build Optimization Explanation

A multi-stage build technique was used in the backend Dockerfile to separate the build phase from the runtime phase.

- **Build Stage:**
 - All required dependencies are installed using `npm install --only=production`.
 - Application files are prepared and organized for deployment.
- **Runtime Stage:**
 - Only the necessary application files and production dependencies are copied into a clean runtime environment.
 - Build tools, temporary files, and development dependencies are left behind in the builder stage.

This approach significantly reduces the overall image size.

Conceptual Flow:

1. Build Stage:

- Install required dependencies
- Build or compile the application

2. Runtime Stage:

- Copy only the necessary application files
- Run the application in a minimal environment

Using this method produces a lighter and more efficient Docker image.

- **Lightweight Base Image**

The backend container uses a lightweight base image: `node:20-alpine`.

Alpine Linux provides a minimal operating system environment, reducing the size of the container image and improving performance.

Benefits of a minimal base image:

- Faster container startup
- Reduced image download time
- Lower storage consumption

The database container uses `postgres:15-alpine` for similar advantages.

- **Use of `.dockerignore`**

A .dockerignore file was used to exclude unnecessary files from the build context, keeping the image clean and speeding up the build process.

Examples of ignored files:

- node_modules
- .git
- .gitignore
- Dockerfile
- npm-debug.log

- **Running Container with a Non-Root User**

For security reasons, the backend container runs with a non-root user instead of root:

```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup  
USER appuser
```

Running as a non-root user minimizes security risks, limiting permissions if the container is compromised.

- **Layer Optimization**

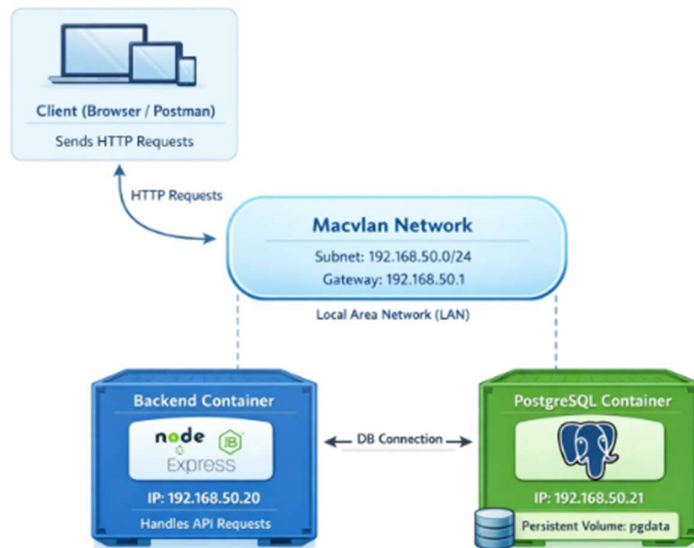
Docker images are built using layers. In this project, related commands were combined, and unnecessary intermediate files were avoided to optimize layers.

Benefits of layer optimization:

- Faster Docker builds
- Better use of build caching
- Reduced final image size

Overall these optimization techniques—multi-stage builds, lightweight base images, .dockerignore, non-root user execution, and layer optimization—produce a smaller, faster, and more secure container image, improving deployment speed and runtime performance in containerized environments.

2. Network Diagram



REPOSITORY LINK : <https://github.com/ananya1192/Containerization-and-Devops/tree/main/container-webapp-project-assign1>

3. Image Size Comparison

Service	Base Image	Final image size
Backend	node:20-alpine	~95 MB
Database	postgres:15-alpine	~75 MB

Multi-stage builds and Alpine-based images significantly reduce image size compared to full images.

4. Macvlan vs Ipvlan Comparison

Feature	Macvlan	Ipvlan
Isolation	High, unique MAC per container	Shares MAC with parent network
IP assignment	One IP per container	Multiple IPs can share a MAC
Use Case	Host communication, static IP	Host communication, static IP
Complexity	Higher; must plan subnet carefully	Lower; simpler setup
Performance	Near-native	Near-native, slightly simpler

- **Macvlan** was used to give containers static IPs and separate host-level visibility for database and backend.
- **Ipvlan** could be used for L3 routing with less MAC isolation, but macvlan suits static IP assignments.